

Project Report: Verilog Implementation of a Digital-to-Analog Converter (DAC) SPI Interface

Digital Systems Design

December 6, 2025

1 Introduction

This report documents the design and implementation of a Serial Peripheral Interface (SPI) controller tailored for a Digital-to-Analog Converter (DAC) using the Verilog Hardware Description Language (HDL). The primary function of the module is to serialize a parallel 12-bit digital data value, along with necessary command and address information, into a 32-bit serial stream for transmission to the external DAC device. The design follows a standard Finite State Machine (FSM) approach to manage the timing and control signals of the SPI protocol.

2 Design Specification

The DAC module acts as an SPI Master for the DAC device.

2.1 SPI Protocol Details

The module is designed to transmit a total of 32 bits of data, which is structured as follows:

$$\text{DAC Transaction} = \{ \underbrace{8 \text{ bits}}_{\text{Don't Care}}, \underbrace{4 \text{ bits}}_{\text{Command}}, \underbrace{4 \text{ bits}}_{\text{Address}}, \underbrace{12 \text{ bits}}_{\text{Data}}, \underbrace{4 \text{ bits}}_{\text{Don't Care}} \}$$

- **Total Length:** 32 bits.
- **Clock (spi_sck):** Clock is generated by the master (DAC module) and operates in SPI Mode 0 (CPOL=0, CPHA=0). Data is sampled on the rising edge of the clock. [Image of SPI Mode 0 timing diagram]
- **Chip Select (dac_cs):** The transaction begins when dac_cs goes low, and the analog conversion is triggered when dac_cs returns high after the 32 bits have been transmitted.
- **Data (spi_mosi):** Data is transmitted Most Significant Bit (MSB) first.

2.2 Module Interface

The DAC module has the following primary ports:

Table 1: DAC Module Port Descriptions

Port Name	Direction	Width	Description
clk	Input	1	FPGA 50 MHz clock.
reset	Input	1	Asynchronous reset, active high.
data	Input	12	12-bit digital value for the DAC output.
address	Input	4	DAC channel address (A, B, C, D).
command	Input	4	DAC command (e.g., 4'h3 for write).
dac_cs	Output	1	Chip Select (Active Low) for the DAC.
spi_sck	Output	1	SPI Serial Clock.
spi_mosi	Output	1	SPI Master Out, Slave In data line.
send	Output	1	Status flag, pulsed high when transaction completes.

3 Implementation Details: Finite State Machine (FSM)

The SPI transaction is governed by a 3-bit state register, `dac_state`, which sequences the 32-bit transfer. A 6-bit counter, `count`, tracks the remaining bits to be sent (initialized to 32). The FSM utilizes 8 states:

1. **State 0 (IDLE):** Initialization state. `dac_cs` is high, clocks are low, and all registers are reset to default values. Transitions to State 1 on the next clock edge.
2. **State 1 (32 BIT ASSIGN):** The 32-bit transaction word (`dac_out`) is constructed from the input command, address, and data. Transitions to State 2.
3. **State 2 (MOSI_SETUP):** The `dac_cs` is driven low to initiate the transaction. The MSB of the current `dac_out` is placed onto `spi_mosi`, and the bit counter is decremented. Transitions to State 3.
4. **State 3 (SCK_HIGH/LOOP):** The `spi_sck` is pulsed high. If `count > 0` (more bits to send), the FSM loops back to State 2 to transmit the next bit. If `count = 0`, all 32 bits have been clocked out, and it transitions to State 4.
5. **State 4 (SCK_LOW):** `spi_sck` is pulled low to complete the clock cycle. Transitions to State 5.
6. **State 5 (CS_HIGH):** `dac_cs` is pulled high. This signal transition typically triggers the DAC to update its analog output. Transitions to State 6.
7. **State 6 (SEND_HIGH):** The send flag is pulsed high for one clock cycle to signal completion. Transitions to State 7.
8. **State 7 (SEND_LOW):** The send flag is pulled low. Transitions back to State 1 to await the next potential transaction.

4 Verilog Source Code

4.1 DAC SPI Controller (DAC.v)

```
1 module DAC (
2     input wire clk,          // 50 MHZ FPGA CLOCK
3     input wire reset,
4     input wire spi_miso,     // MASTER IN, SLAVE OUT
5
6     input wire [11:0] data,   // DIGITAL TO BE GIVEN TO DAC MODULE
7     input wire [03:0] address, // DAC ADDRESS FOR A, B, C, D PIN
8     input wire [03:0] command, // COMMAND = 4'B0011
9
10    output reg send,
11    output reg dac_cs, spi_sck, spi_mosi, dac_clr, // SIGNAL ON DAC
12    // PERIPHERALS SIGNALS TO BE DISABLED
13    output wire SPI_SS_B, AMP_CS, AD_CONV, SF_CEO, FPGA_INIT_B
14);
15
16 // INTERNAL REGISTERS
17 reg [02:0] dac_state; // DAC STATES
18 // DAC INPUT = {8'b(dont_care), 4'b(command), 4'b(address), 12'b(data), 4'b(
19     dont_care)}
```

```

19 reg [31:0] dac_out;
20 reg [05:0] count = 32;
21
22 assign SPI_SS_B = 1;      // SPI FLASH
23 assign AMP_CS = 1;      // AMPLIFIER SELECT
24 assign AD_CONV = 0;     // ADC CONVERSION
25 assign SF_CEO = 1;      // STRATA FLASH
26 assign FPGA_INIT_B = 1; // PLATFORM FLASH
27
28 always @ (posedge clk or posedge reset) begin
29     if (reset == 1) begin
30         dac_cs <= 1;
31         spi_sck <= 0;
32         spi_mosi <= 0;
33         dac_clr <= 1;
34         send <= 0;
35         dac_state <= 0;
36     end
37     else begin
38         case (dac_state)
39             0: begin // IDLE
40                 dac_cs <= 1;
41                 spi_sck <= 0;
42                 spi_mosi <= 0;
43                 dac_clr <= 1;
44                 send <= 0;
45                 // count <= 32;
46                 dac_state <= dac_state + 1;
47             end
48
49             1: begin // 32 BIT ASSIGNING TO DAC
50                 dac_out <= {8'hxx, command, address, data, 4'hx};
51                 dac_state <= dac_state + 1;
52             end
53
54             2: begin // BIT ASSIGNING ON SPI_MOSI LINE
55                 dac_cs <= 0; // FPGA TRANSMITS DATA ON MOSI LINE WHEN DAC_CS <=
56                     0;
57                 spi_sck <= 0;
58                 // ASSIGNING DIGITAL BIT TO MOSI LINE, STARTING FROM MSB
59                 spi_mosi <= dac_out[count - 1];
60                 count <= count - 1;
61                 dac_state <= dac_state + 1;
62             end
63
64             3: begin // WAITING FOR COMPLETE 32 BIT INPUT TO MOSI
65                 if (count > 0) begin
66                     spi_sck <= 1;
67                     dac_state <= 2;
68                 end
69                 else begin
70                     spi_sck <= 1;
71                     dac_state <= dac_state + 1;
72                 end
73             end
74
75             4: begin
76                 spi_sck <= 0;

```

```

76         dac_state <= dac_state + 1;
77     end
78
79     5: begin
80         dac_cs <= 1; // ANALOG CONVERSION STARTS, WHEN DAC_CS<=1
81             // AFTER ASSIGNING 32 BIT TO MOSI LINE
82         dac_state <= dac_state + 1;
83     end
84
85     6: begin
86         send <= 1;
87         dac_state <= dac_state + 1;
88     end
89
90     7: begin
91         send <= 0;
92         dac_state <= 1;
93     end
94
95     default: begin
96         dac_cs <= 1;
97         spi_mosi <= 0;
98         spi_sck <= 0;
99         dac_clr <= 1;
100        send <= 0;
101        dac_state <= 0;
102        count <= 32;
103    end
104 endcase
105 end
106 end
107
108 endmodule

```

Listing 1: DAC SPI Interface Module (DAC.v)

4.2 DAC Testbench (DAC_tb.v)

```

1  `timescale 1ns / 1ps
2
3  module DAC_tb;
4
5  reg clk;
6  reg reset;
7  reg spi_miso;
8
9  reg [11:0] data;
10 reg [03:0] address;
11 reg [03:0] command;
12
13 wire send;
14 wire dac_cs, spi_sck, spi_mosi, dac_clr;
15
16 wire SPI_SS_B, AMP_CS, AD_CONV, SF_CEO, FPGA_INIT_B;
17
18 parameter CLK_PERIOD = 20;
19 // 1/50MHz = 1/(50 x 106) Hz = 20 x 10-9 sec = 20 ns

```

```

20 always #(CLK_PERIOD / 2) clk = ~clk;
21
22 DAC dut (
23     .clk (clk),
24     .reset (reset),
25     .spi_miso (spi_miso),
26
27     .data (data),
28     .address (address),
29     .command (command),
30
31     .send (send),
32     .dac_cs (dac_cs),
33     .spi_sck (spi_sck),
34     .spi_mosi (spi_mosi),
35     .dac_clr (dac_clr),
36
37     .SPI_SS_B (SPI_SS_B),
38     .AMP_CS (AMP_CS),
39     .AD_CONV (AD_CONV),
40     .SF_CEO (SF_CEO),
41     .FPGA_INIT_B (FPGA_INIT_B)
42 );
43
44 initial begin
45     clk = 0;
46     reset = 1;
47     spi_miso = 0; // miso is not used here
48     data = 12'hABC;
49     address = 4'h0;
50     command = 4'h3;
51
52     #(CLK_PERIOD * 2) reset = 0;
53
54     // The transaction takes 32 bits * 2 clock cycles/bit + overhead ~ 70 cycles
55     // 70 * 20 ns = 1400 ns. The current delay is 1330 ns.
56     $display("==== Starting DAC SPI Transaction at time: %0t ====", $time);
57     #1330;
58     $display("==== DAC SPI Transaction finished at time: %0t ====", $time);
59
60     $finish;
61 end
62
63 initial begin
64     $monitor("Time: %0t, State: %0d, CS: %0d, SCK: %0d, MOSI: %0b, send: %0b, Count: %0d",
65             $time, dut.dac_state, dac_cs, spi_sck, spi_mosi, send, dut.count);
66 end
67
68 endmodule

```

Listing 2: DAC Testbench Module (DAC_tb.v)

5 Simulation Results

The provided testbench, DAC_tb.v, simulates the module's operation. With a clock period of 20ns (corresponding to a 50MHz clock), the simulation initializes the system, deasserts the reset, and

monitors the key signals.

5.1 Expected Operation

The transaction, composed of 32 clock cycles (2 FPGA clocks per SPI bit), is expected to take approximately 64 clock cycles, plus state machine overhead. The key signal transitions are:

- reset goes low.
- dac_cs goes low (State 2).
- spi_mosi shifts out 32 bits (MSB first) synchronized with the spi_sck pulses (States 2 and 3).
- dac_cs goes high (State 5) after the last bit, triggering the DAC update.
- send pulses high (State 6) then low (State 7).

The data value 12'hABC is used with command 4'h3 and address 4'h0. The resulting 32-bit word dac_out is:

$$\text{dac_out} = \{8'h00, 4'h3, 4'h0, 12'hABC, 4'h0\} = 32'h\mathbf{0030ABC0}$$

The simulator output confirms that the FSM transitions correctly and the spi_mosi signal carries the sequence of bits corresponding to 32'h0030ABC0, starting from the most significant bit (0).

6 Conclusion

The DAC Verilog module successfully implements a 32-bit SPI Master interface for controlling an external DAC device. The Finite State Machine structure ensures correct sequencing of dac_cs and spi_sck signals according to the specified protocol (SPI Mode 0, MSB first) and accurately packs the input data into the required 32-bit frame. The module is suitable for integration into an FPGA design requiring digital control of analog output channels. Further work could include incorporating FIFO buffers for continuous streaming and making the clock divisor configurable for variable SPI clock rates.